

Resumen Estructurado — Orquestación y Despliegue de Agentes

Documento optimizado para chunking y sistemas RAG. El contenido ha sido reorganizado en bloques conceptuales claros y reutilizables.

1. Del notebook a producción

Un agente funcional en local no implica que esté preparado para producción. El despliegue real requiere escalabilidad, seguridad, observabilidad y tolerancia a fallos.

Notebook / Demo	Producción
Funciona en local	Accesible vía API/web
API keys visibles	Secretos gestionados
Un usuario	Concurrencia y escalado
Reinicio manual	Reintentos automáticos
Sin logs	Monitorización y métricas

2. Retos principales en despliegue

- Latencia y coste: llamadas LLM lentas y costosas.
- Fiabilidad: timeouts, errores y fallos del modelo.
- Seguridad: protección de API keys y sanitización de inputs.
- Necesidad de caching y control de concurrencia.

3. Arquitectura de agentes con LangGraph

LangGraph organiza agentes mediante nodos, estados y rutas condicionales. El flujo de ejecución se modela como un grafo.

- **Nodo LLM:** razonamiento y generación.
- **Nodo RAG:** recuperación de contexto.
- **Estado compartido** entre nodos.
- **Router condicional** para decidir flujo.
- **Checkpointing** para persistencia.

4. Patrones de orquestación

Existen diferentes estrategias para coordinar agentes y herramientas.

- **Pipeline:** agentes en secuencia.
- **Supervisor:** agente orquestador delega tareas.
- **Agente único:** múltiples nodos en un mismo grafo.
- El patrón recomendado para proyectos simples es agente único.

5. Gestión del estado y memoria

Los LLMs no recuerdan conversaciones entre llamadas, por lo que se requiere persistencia externa.

- **MemorySaver:** memoria en RAM para demos.
- **SqliteSaver:** persistencia local.
- **PostgresSaver:** producción multiusuario.
- **thread_id:** identifica conversaciones independientes.

6. Manejo de errores y reintentos

Las aplicaciones productivas deben manejar errores de red, timeouts y respuestas inválidas.

- **Uso de reintentos automáticos.**
- **Backoff exponencial.**
- **Fallbacks** cuando el LLM falla.
- **Bibliotecas comunes:** tenacity.

7. Observabilidad y monitorización

LangSmith permite monitorizar agentes LangChain/LangGraph en tiempo real.

- Seguimiento de trazas y decisiones.
- Monitorización de latencia.
- Control de consumo de tokens.
- Registro de errores y feedback de usuarios.

8. Opciones de despliegue

Opción	Ventajas	Limitaciones
Local / Notebook	Simple y sin coste	Sin acceso externo
Streamlit Cloud	Deploy rápido y URL pública	Cold starts y recursos limitados
FastAPI + Cloud Run	Escalado automático y producción	Mayor complejidad

9. FastAPI como capa de servicio

FastAPI permite exponer agentes como APIs REST consumibles desde aplicaciones web o móviles.

- Endpoints asíncronos.
- Integración sencilla con LangGraph.
- Uso de modelos Pydantic.
- Compatible con despliegue cloud.

10. Seguridad y control de costes

La seguridad y optimización de costes son elementos críticos en sistemas con LLMs.

- No hardcodear API keys.
- Uso de .env y Secret Managers.
- Rate limiting por usuario.
- Cache de respuestas frecuentes.
- Limitar max_tokens y llamadas paralelas.
- Sanitizar inputs para evitar prompt injection.

11. Arquitectura end-to-end

Una arquitectura completa de agentes combina frontend, APIs, memoria, observabilidad y bases vectoriales.

- Cliente: Streamlit, Postman o servicios externos.
- API Gateway: FastAPI + autenticación.
- Agente: LangGraph + RAG + Gemini.
- Persistencia: SQLite o Postgres.
- Vector DB: ChromaDB.
- Observabilidad: LangSmith y logs.

12. Proyecto final del curso

El proyecto consiste en construir un agente experto con Gemini, RAG y memoria conversacional.

- Base de conocimiento en ChromaDB.
- Embeddings de Gemini.
- LangGraph con memoria.
- Notebook funcional con ejemplos.
- README justificando prompts y arquitectura.

13. Criterios de evaluación

La evaluación se centra en calidad técnica, integración RAG, memoria y documentación.

- Correcta indexación documental.
- Precisión de recuperación.
- Uso adecuado de prompts.
- Memoria funcional.
- Código limpio y documentado.
- Presentación funcional con demo.

14. Conclusión

La orquestación y despliegue de agentes transforma prototipos de IA en aplicaciones reales y escalables. Conceptos como LangGraph, FastAPI, observabilidad, memoria persistente y control de costes son esenciales para construir sistemas robustos basados en LLMs y RAG.